Starburst

Trino Query Plan Analysis *Starburst Workshops*

A hands-on webinar
with Lester Martin from DevRel

Connection before content

Lester Martin – https://linktr.ee/lestermartin

- Developer Relations @ Starburst
  - Blogging & forums
  - Webinars & videos
  - User groups & events
  - Training & tutorials
- 30+ years of technology experience
  - Started journey on TRS-80 Model III
  - Played most roles, but a programmer at my core
  - ½ career in OLTP and ½ in data analytics
  - Decade+ of "big data" experience to include
    - Trino/Starburst, Hadoop, Hive, Spark
    - NiFi, Kafka, Storm, Flink
    - HBase, MongoDB

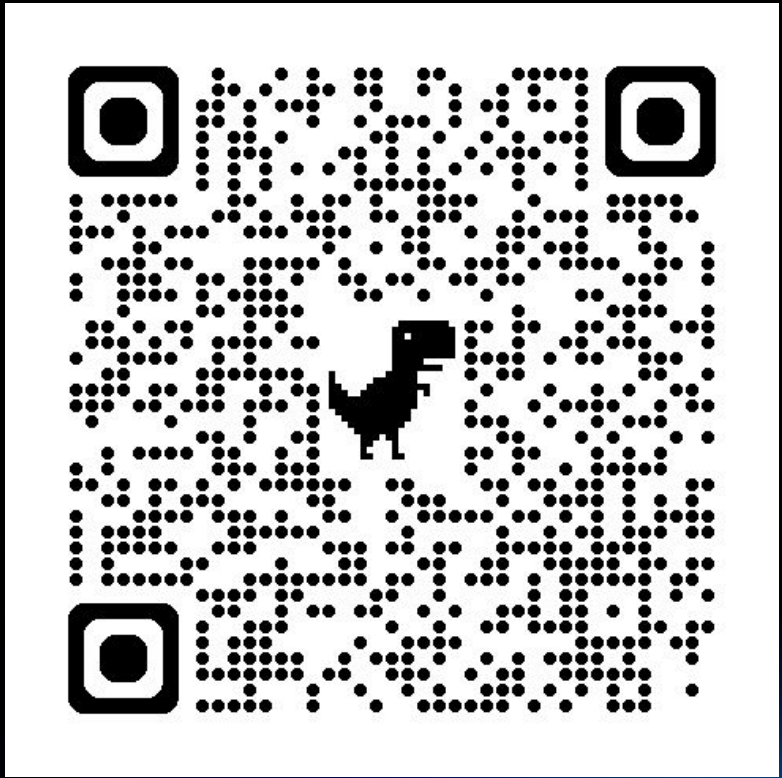
devrel@starburst.io

Starburst

Environment setup

Setup and instructions located on Github

https://github.com/starburstdata/devrel/blob/main/workshops/query-plans/INSTRUCTIONS.md



Environment

Using Starburst Galaxy and S3

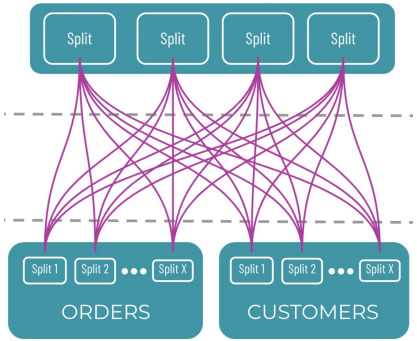
Exercises

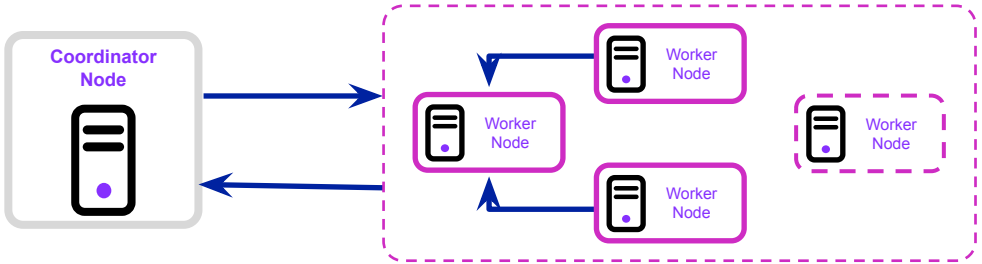
All SQL needed avail for download

Massively Parallel Processing (MPP)

Query the data lake with Starburst and Trino

Starburst





Starburst

Hands-on

Validate env setup

Parallel processing

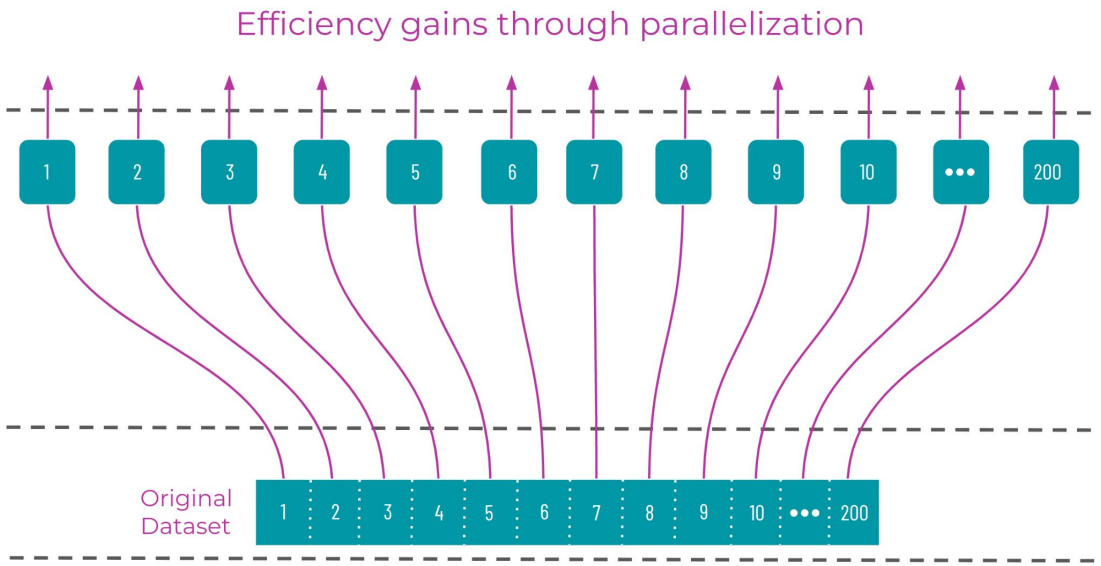
Divide and conquer

Understanding splits

Parallelization divides a single workload into independent units running concurrently

Process efficiently

- Imagine that you have the same 20GB of data as before
- Now imagine that you split this data into equal parts and divided the workload evenly
- With 200 splits, each with 200K records, the 10K/sec processing speed yields 20 secs/split

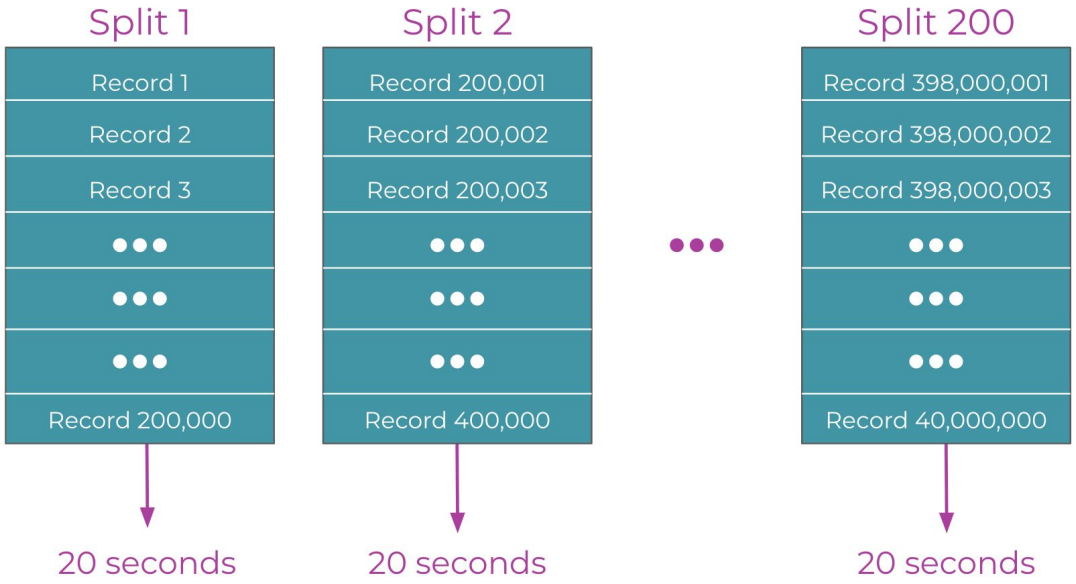


Understanding splits

Parallelization divides a single workload into independent units running concurrently

Work simultaneously

- Each split can be processed in parallel
- 20 seconds for all splits at the same time - instead of 4000 seconds if processed sequentially
- Possible if sufficient resources to run all splits at the same time

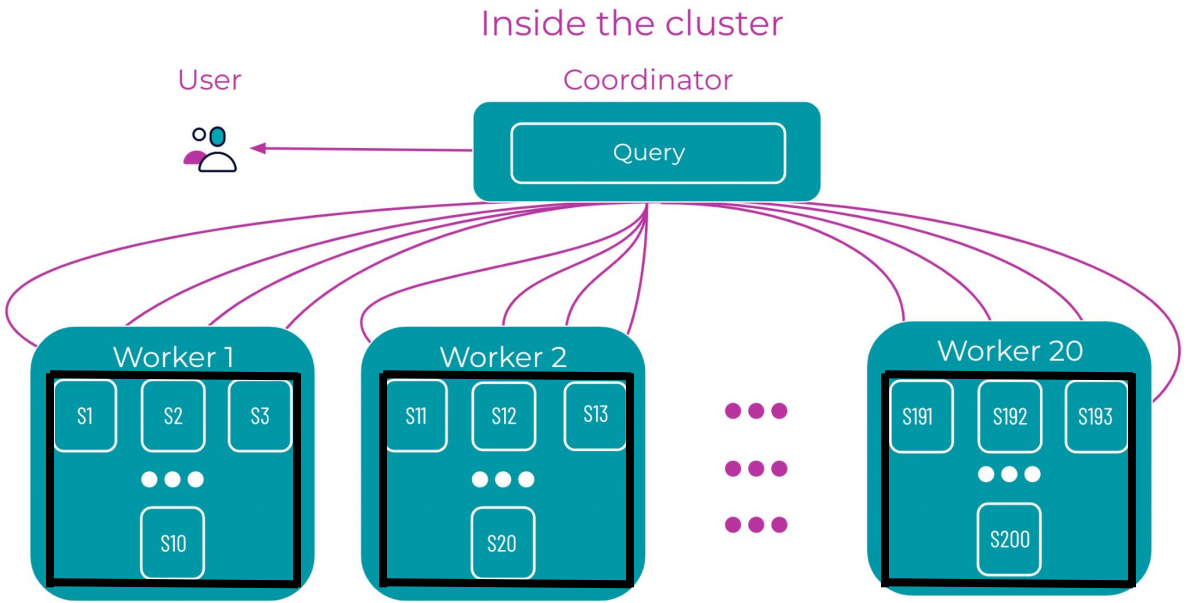


Starburst

Visualizing tasks

Tasks operate on splits

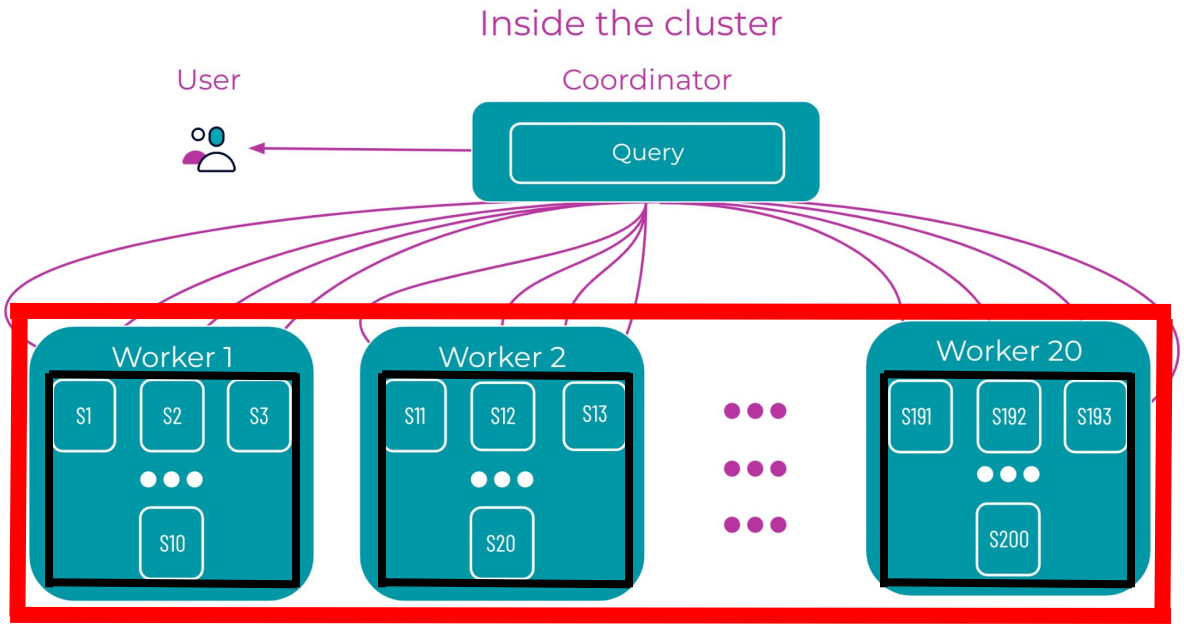
- Each worker participating in the parallelized processing of multiples splits is assigned a single task.
- Each task (again, one per worker) is assigned 1+ Splits.



Visualizing tasks, stages

Tasks operate on splits & a stage includes all those tasks

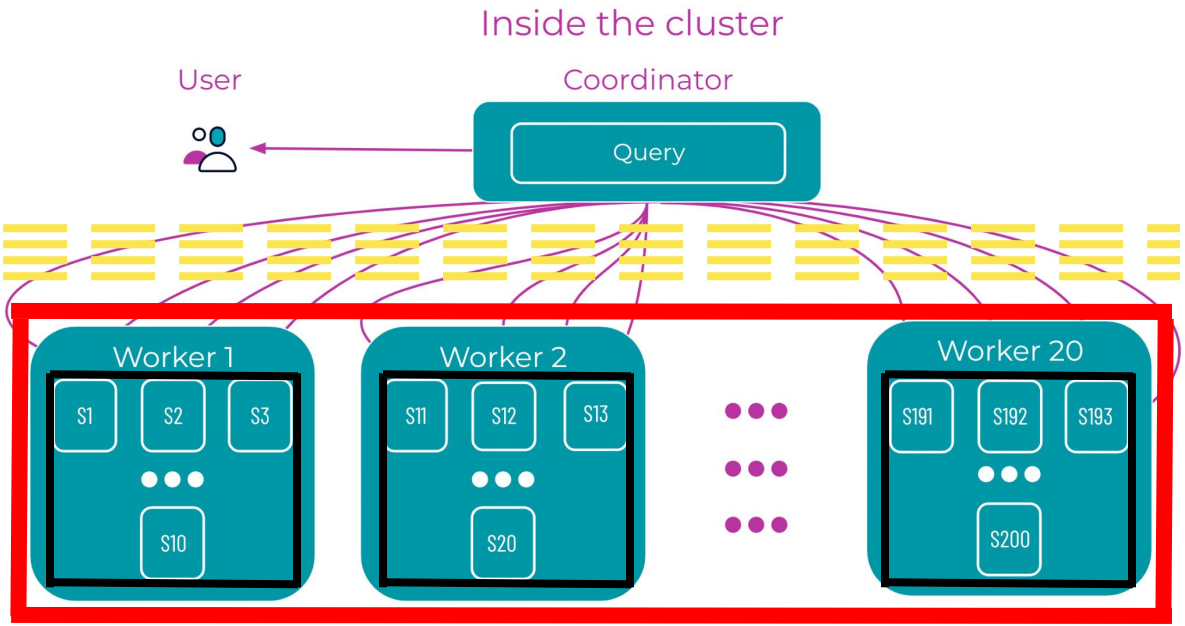
- Each worker participating in the parallelized processing of multiples splits is assigned a single task.
- Each task (again, one per worker) is assigned 1+ Splits.
- All the tasks across the cluster performing the same kind of work, just on different splits, are collectively called a stage.



Visualizing tasks, stages and exchanges

Tasks operate on splits & a stage includes all those tasks & stages exchange data

- Each worker participating in the parallelized processing of multiples splits is assigned a single task.
- Each task (again, one per worker) is assigned 1+ Splits.
- All the tasks across the cluster performing the same kind of work, just on different splits, are collectively called a stage.
- A stage transmits its results back to the coordinator, or another stage, via an exchange of data.



Starburst

Hands-on

Single-stage plans

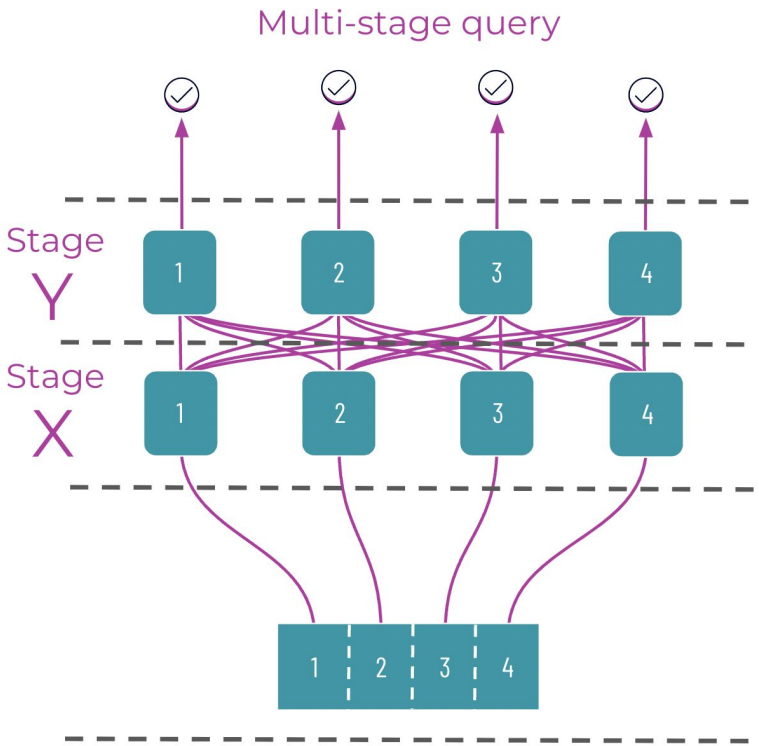
Parallel processing

Beyond single stage queries

Multi-stage queries

Multi-stage queries

- Consider a 2 stage query.
- Two separate operation pipelines are needed.
- The first pipeline is processed in Stage X.
  - The system splits the data set.
  - Processes them in parallel.
- The system addresses the second pipeline in Stage Y.
  - The output of Stage X is exchanged to become the input of Stage Y.
  - This reordered dataset has splits, too.
  - Parallelization is employed a second time.
- After all of these operations are complete, the results of the entire query are returned to the user.



Sorting results with `ORDER BY`

```
SELECT Col_1, Col_2
  FROM table
 ORDER BY Col_1, Col_2 DESC
```

1. Scanning stage

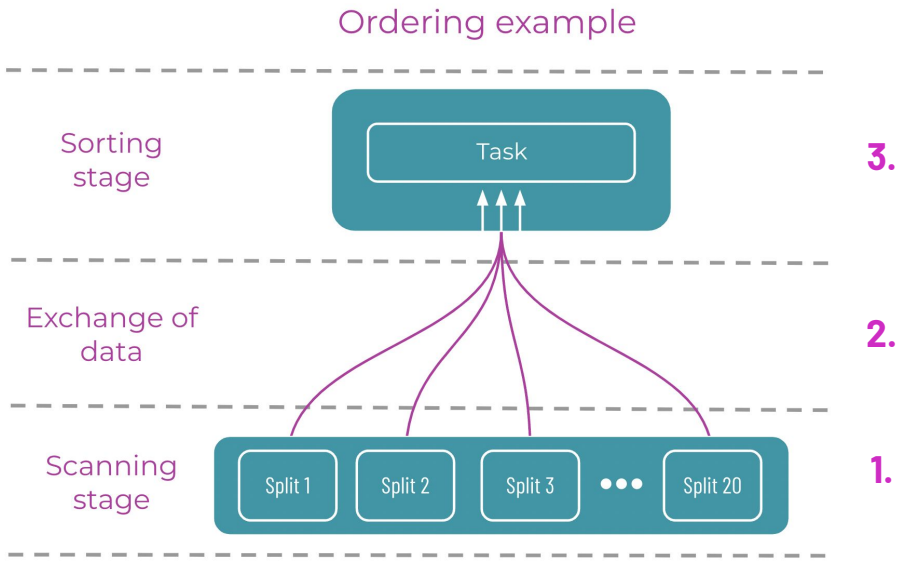
- Uses the `SELECT` command to select all of the data requested.
- This stage is divided into many splits that are operated on in parallel across the workers.

2. Exchange of data

- When each task is complete, the results are sent from each worker to the a single split dataset.

3. Sorting stage

- After the unsorted data has been received, the results will be sorted according to the `ORDER BY` clause.



Aggregation with GROUP BY

```
SELECT Col_5, max(Col_3) FROM table_name
 WHERE Col_3 IN ('value_A', 'value_B', value_C')
   AND Col_5 LIKE ('bonus%')
 GROUP BY Col_5
```

1. Scanning stage

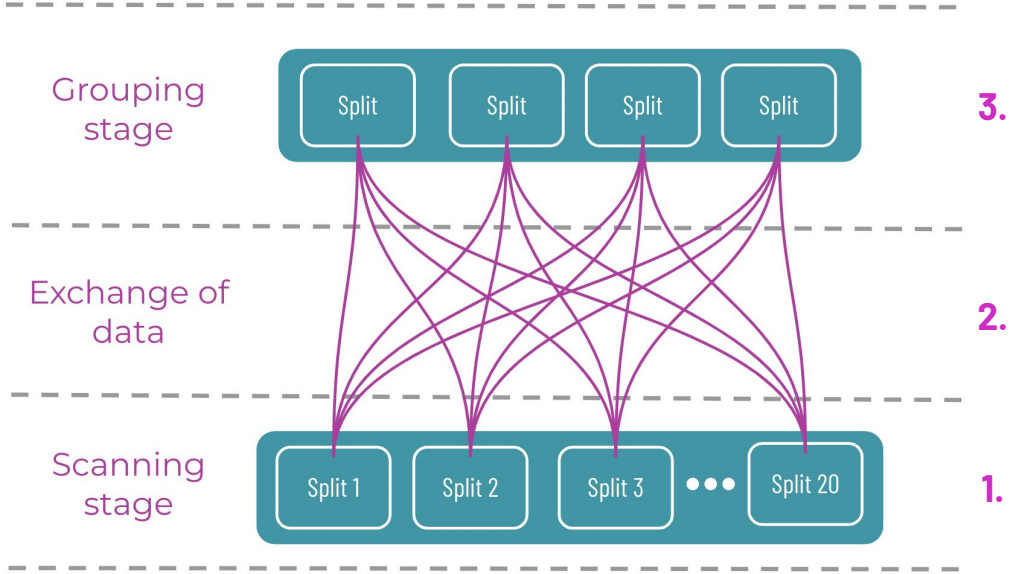
- `SELECT` command executes normally.
- Searches the database for all data that matches the query criteria.

2. Exchange of data

- Any recurrences of a given value are hashed on the aggregated column.
- For example, `Col_5`.

3. Grouping stage

- Massive transfer of hashed data.
- Collapses duplicate fields of values into individual, single values.



Partitioned joins

Joining

- Parallelism also plays a role in querying multiple tables.
- Relational databases separate different kinds of data into different tables:
  - Providing organization
  - Preventing duplication.
- For example, imagine a database records both customer data and order data.
  - Separate data types in separate tables.
  - How do you gather insights across multiple tables?
  - Use a Join.
- There are many varieties of joins from the SQL syntax (ex: inner, outer, etc).
- **The default underlying implementation is called a partitioned join**.

```
SELECT column_name
FROM table_name_1
INNER JOIN table_name_2
ON table_name_1.column_name = table_name_2.column_name;
```

Partitioned joins

1. Scanning stages

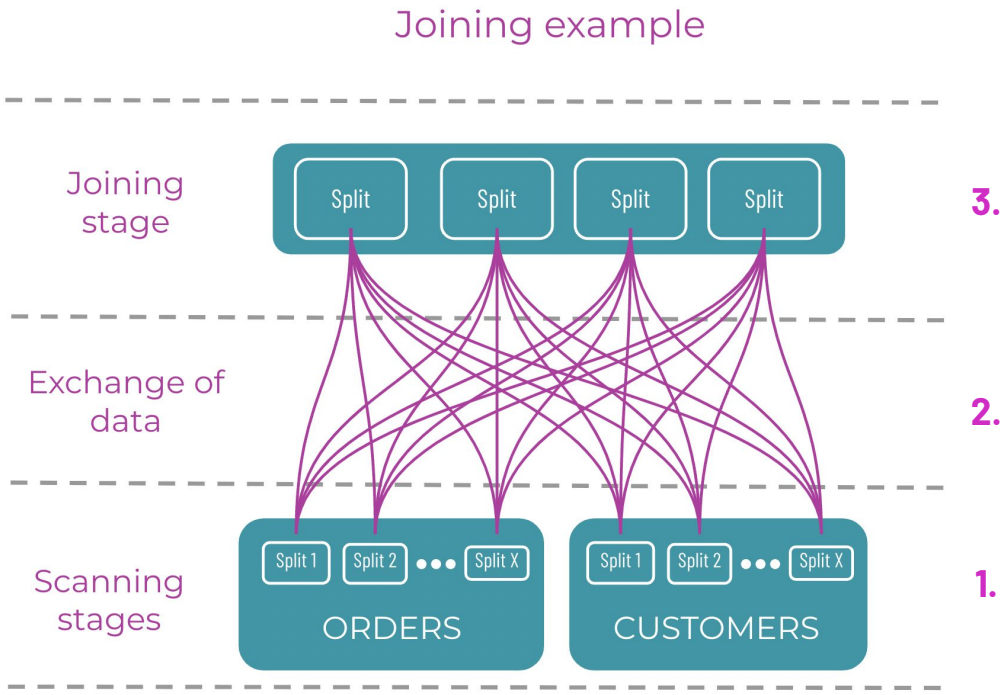
- Two independent scanning stages.
- `SELECT` command searches both the ORDERS and CUSTOMERS tables.

2. Exchange of data

- Join key providing a common point of overlap.
- Acts as a bridge between ORDERS and CUSTOMER tables.
- Data is exchanged in parallel, driven by a hash of the join key.

3. Joining stage

- The joined data is reassembled using the join key.
- Processing operates in parallel.



Broadcast joins

Useful for small datasets joined with larger ones

- Partitioned joins are a great tool connecting data between two tables.
- But they're inefficient:
  - Two stages to scan the data.
  - Costly data exchange.
  - A third stage to join the data.
- Broadcast joins fix this, by:
  - Making use of asymmetries in the size of the tables.
  - Transferring a copy of the smaller table to the workers
- In this way:
  - Each worker has a cached copy of the smaller table.
  - Allows for the construction of a join within each split.
- **No SQL changes/hints are needed** – Starburst decides on this implementation if the smaller table can fit in memory.

```
SELECT column_name
FROM table_name_1
INNER JOIN table_name_2
ON table_name_1.column_name = table_name_2.column_name;
```

Broadcast joins

1. CUSTOMERS table scan

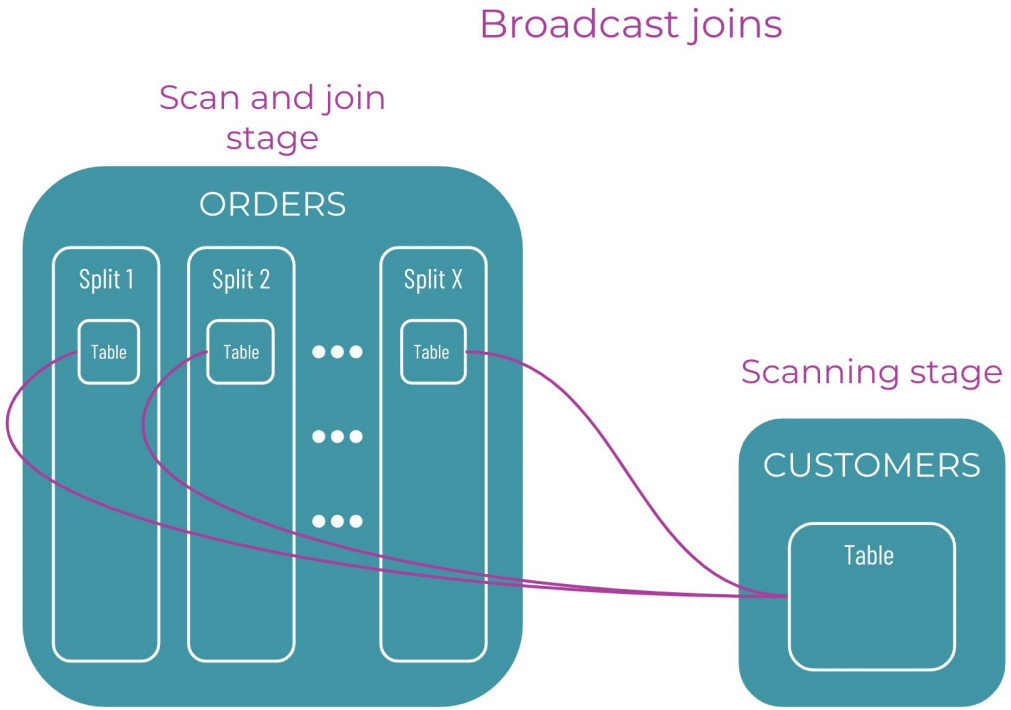
- The CUSTOMERS table is small.
- This makes a broadcast join possible.

2. CUSTOMERS sent to workers

- Broadcast join sends the entire CUSTOMERS table will be sent to all of the workers in the cluster.
- Table loaded into each worker's memory.

3. Joining stage

- ORDERS table is scanned in parallel.
- The join happens in parallel, just as it reads a record from ORDERS it joins in the required data from the matching CUSTOMERS table that is in memory.



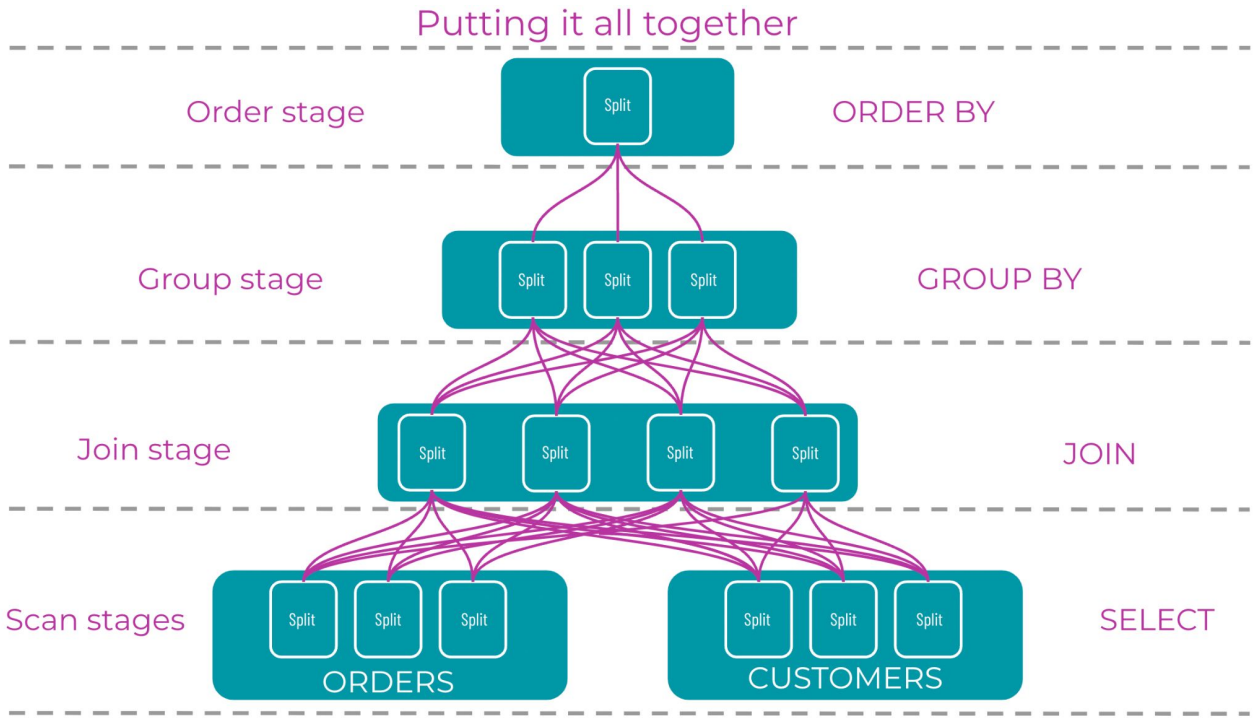
Putting it all together

1. Scanning stages

- The scan stages are executed first.
- Use `SELECT` on both the ORDERS and CUSTOMERS tables.
- Processing completed in parallel.

2. Joining stage

- ORDERS are joined with CUSTOMERS.
- Tasks completed in parallel.



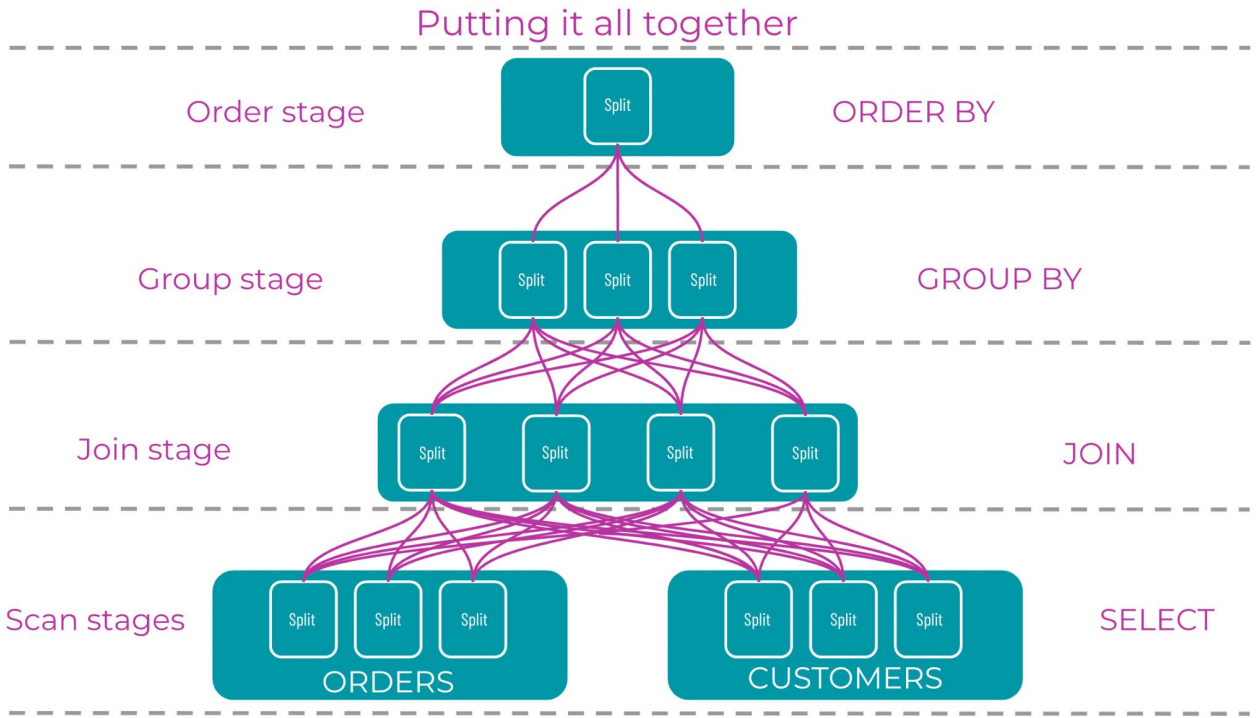
Putting it all together

3. Grouping stage

- `GROUP BY` scans for any recurring values.
- Workload divided among the workers.
- Completed in parallel.

4. Order stage

- `ORDER BY` lists the results in ascending or descending list.
- Processed in parallel.
- Output is gathered by the coordinator and returned to the user.



Starburst

Hands-on

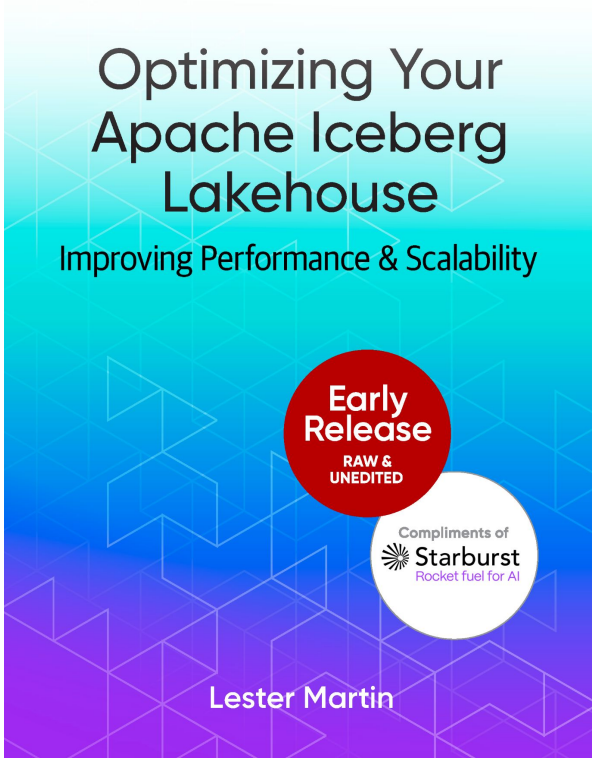
Multi-stage plans

What are my next steps?

- https://devcenter.starburst.io/
- https://www.starburst.io/community/forum/
- https://www.starburst.io/learn/events-webinars/
- https://lestermartin.blog/2025/04/22/trino-query-plan-analysis-video-series/

Even more shameless self-promotion…

Optimizing Your Apache Iceberg Lakehouse



Demo artifacts at https://github.com/starburstdata/devrel/tree/main/workshops/query-plans

Starburst

Thank you!

devrel@starburst.io